Zibo Payments & Credits: Implementation Plan

Nov 11, 2026 · @SATYAJIT PUJAPANDA

Payments, credits and paywalls must stay correct under retries, concurrency and crashes. Build the ledger first, then gate AI and content, then add Razorpay, and treat the 22 listed races and the test matrix as the definition of done.

Goals and invariants

Payments and credits must stay correct under retries, concurrent requests, duplicate webhooks and crashes. Eight invariants define "correct". Every design choice below enforces one of them, and every test in the matrix checks one. This plan replaces sections 3 to 6 of the master spec and keeps its product decisions: 50 free credits once, PRO at 2,500 credits a month, nine metered AI actions, and a preview of premium apps.

#	Invariant	Enforced by
I1	`creditsBalance` always equals the sum of that user's `CreditLedger.amountDelta`	One DB transaction per balance change, ledger row inserted in the same transaction, nightly verifier
I2	Balance never goes below zero	`CHECK (creditsBalance >= 0)` plus a conditional `UPDATE ... WHERE creditsBalance >= cost`
I3	Every credit-moving or tier-moving operation happens at most once	Deterministic idempotency key, unique-constrained and scoped by user
I4	Razorpay is the source of truth for payment facts, our DB for credits	No grant from a browser callback; fulfilment only from verified webhooks or the reconciler
I5	`tier` is derived from subscription state plus an admin override, never set freehand	One `recomputeTier(userId)` function, called by every handler
I6	Locked screen content never reaches a non-PRO user: images, analysis, or AI output built from them	One `canViewScreen` policy used by every route and every AI context loader

#	Invariant	Enforced by
I7	Every Claude call is paid for up front, and its real USD cost is logged even when credits are refunded	Reserve, then settle or release; `AiRequestLog` written on every outcome
I8	Any missed or failed event repairs itself	Reconciler, sweeper and verifier, with alerts when they find drift

Architecture and sources of truth

Every fact has exactly one owner. Everything else is a copy that a worker can rebuild from the owner, which is what makes the reconciler safe to run at any time.

Fact	Owner	Local copy	Repaired by
A payment succeeded, failed or was refunded	Razorpay	`PaymentTransaction`	Webhook; reconciler fetches the payment or invoice
Subscription status and billing period	Razorpay	`UserSubscription`	Webhook; reconciler fetches the subscription
Credit balance	`CreditLedger` (append-only)	`ClientUser.creditsBalance`	Nightly verifier recomputes the sum and alerts on drift
Tier	Derived from subscription plus admin override	`ClientUser.tier`	`recomputeTier(userId)` after every change, and hourly for entitlements about to expire
Whether a request has been paid for	`CreditReservation`	None	Sweeper releases stuck reservations
Whether a screen is locked	Server policy `canViewScreen`	None; the client never decides	Not applicable

Three rules follow from the table.

1. **Only `CreditService.apply()` writes `creditsBalance`.** The AI gate, the webhook handlers, the sweeper and the admin endpoint all call it. A CI check fails the build if any other file writes that column.

2. **The browser never grants anything.** The checkout callback only tells the UI to start polling `GET /billing/status`. Credits and tier change only when a verified webhook or the reconciler says so.
3. **Handlers do not set `tier`.** They update `UserSubscription` and call `recomputeTier`, so the result is the same whichever event arrives first.

Final data model

The master spec's schema needs seven structural changes before any code is written. Each one removes a specific failure.

Change	Failure it removes
`creditsBalance` defaults to 0 and the welcome grant becomes a ledger row	The nightly verifier would flag every user, since balance 50 never equals ledger sum 0
New `CreditReservation` table, unique on `(userId, requestKey)`	The sweeper had nothing to sweep, and client retries could double-charge
`PlanPrice` per currency, replacing `priceInr` and `priceUsd`	A Razorpay plan carries one currency, so one `razorpayPlanId` cannot serve INR and USD
`UserSubscription` gets nullable `currentPeriodEnd`, `entitledUntil` and `lastEventAt`; `userId` is no longer unique	Period end is unknown before the first charge; stale events could overwrite newer state; users could never resubscribe
`tierOverride` on `ClientUser`	An admin's VIP toggle was overwritten by the next webhook
`PaymentWebhookFailure` dead-letter table	Events that fail for more than Razorpay's 24-hour retry window were lost
`AdminAuditLog`, `Decimal` for cost, `Restrict` instead of `Cascade` on money tables	Untraceable admin edits, float drift in cost reports, deleting a user erasing financial history

```
enum LedgerType {
  FREE_WELCOME_GRANT
  MONTHLY_SUBSCRIPTION_REFILL
  ROLLOVER_EXPIRY
  CREDIT_PACK_PURCHASE
```

```
  AI_QUERY_DEBIT
  SYSTEM_REFUND
  PAYMENT_REFUND_CLAWBACK
  ADMIN_MANUAL_GRANT
  ADMIN_MANUAL_DEDUCTION
  MIGRATION_BACKFILL
}

enum ReservationStatus {
  RESERVED
  SETTLED
  RELEASED
}

enum SubscriptionStatus {
  INCOMPLETE   // created or authenticated, first charge not confirmed
  ACTIVE
  PAST_DUE     // Razorpay state pending: charge failed, retries running
  HALTED       // retries exhausted
  PAUSED
  CANCELLED
  COMPLETED
  EXPIRED
}

model ClientUser {
  // existing fields stay
  creditsBalance     Int       @default(0)      // welcome grant arrives as a
ledger row
  tier               UserTier  @default(FREE)   // cache, written only by
recomputeTier()
  tierOverride       UserTier?                  // admin VIP or team accounts
  tierOverrideUntil  DateTime?
  tierOverrideReason String?
  welcomeGrantedAt   DateTime?
}

model CreditReservation {
  id            String            @id @default(uuid())
  userId        String
  user          ClientUser        @relation(fields: [userId], references:
[id], onDelete: Restrict)
  requestKey    String            // client Idempotency-Key, one per user
```

```
action
  actionType    String
  payloadHash   String            // same key with a different payload is
rejected
  cost          Int
  status        ReservationStatus @default(RESERVED)
  releaseReason String?
  expiresAt     DateTime          // request hard timeout plus a safety
margin
  createdAt     DateTime          @default(now())
  settledAt     DateTime?

  @@unique([userId, requestKey])
  @@index([status, expiresAt])
}

model CreditLedger {
  id             String     @id @default(uuid())
  userId         String
  user           ClientUser @relation(fields: [userId], references: [id],
onDelete: Restrict)
  amountDelta    Int
  balanceAfter   Int
  type           LedgerType
  idempotencyKey String     @unique   // namespaced: "debit:<userId>:
<requestKey>", "pay:<razorpayPaymentId>"
  reservationId  String?
  paymentId      String?
  reason         String?
  adminId        String?
  createdAt      DateTime   @default(now())

  @@index([userId, createdAt])
}

model PlanPrice {
  id             String           @id @default(uuid())
  planId         String
  plan           SubscriptionPlan @relation(fields: [planId], references:
[id])
  currency       String           // "INR" | "USD"
  amountMinor    Int              // paise or cents
  razorpayPlanId String           @unique
```

```
  isActive       Boolean          @default(true)

  @@unique([planId, currency, amountMinor])
}

model UserSubscription {
  id                 String             @id @default(uuid())
  userId             String
  user               ClientUser         @relation(fields: [userId],
references: [id], onDelete: Restrict)
  planId             String
  priceId            String
  razorpaySubId      String             @unique
  status             SubscriptionStatus @default(INCOMPLETE)
  razorpayStatus     String?            // raw value kept for debugging
  currentPeriodStart DateTime?
  currentPeriodEnd   DateTime?          // unknown until the first charge
  entitledUntil      DateTime?          // access boundary, includes any
grace
  cancelAtPeriodEnd  Boolean            @default(false)
  lastEventAt        DateTime?          // newest applied event timestamp,
guards out-of-order delivery
  createdAt          DateTime           @default(now())
  updatedAt          DateTime           @updatedAt

  @@index([userId])
  @@index([status])
}

model PaymentWebhookEvent {
  id          String   @id @default(uuid())
  eventId     String   @unique       // x-razorpay-event-id
  event       String
  entityId    String?
  payload     Json
  processedAt DateTime @default(now())
}

model PaymentWebhookFailure {
  id         String    @id @default(uuid())
  eventId    String
  event      String?
  payload    Json
```

```
  error      String
  attempts   Int       @default(1)
  resolvedAt DateTime?
  createdAt  DateTime  @default(now())

  @@index([resolvedAt])
}

model AdminAuditLog {
  id           String   @id @default(uuid())
  adminId      String
  action       String   // "credits.adjust", "tier.override",
"app.premium_toggle"
  targetUserId String?
  before       Json?
  after        Json?
  reason       String
  requestKey   String   @unique
  createdAt    DateTime @default(now())
}
```

Prisma cannot express the following guards, so they go in a raw SQL migration:

```
ALTER TABLE "ClientUser" ADD CONSTRAINT credits_non_negative CHECK
("creditsBalance" >= 0);
ALTER TABLE "CreditLedger" ADD CONSTRAINT ledger_balance_non_negative CHECK
("balanceAfter" >= 0);
ALTER TABLE "CreditReservation" ADD CONSTRAINT reservation_cost_positive
CHECK (cost > 0);

-- at most one live subscription per user; a double-click on Start With PRO
cannot create two
CREATE UNIQUE INDEX one_live_subscription_per_user
  ON "UserSubscription" ("userId")
  WHERE status IN ('INCOMPLETE', 'ACTIVE', 'PAST_DUE', 'PAUSED');

-- the ledger is append-only for the application role
REVOKE UPDATE, DELETE ON "CreditLedger" FROM app_role;
```

Optional hardening: a deferred constraint trigger that checks, at commit, that the newest ledger row's `balanceAfter` equals `creditsBalance`. It makes invariant I1 a database guarantee rather than a convention.

Credit engine

Every balance change goes through one function that runs inside a single database transaction. That makes three things impossible: a negative balance, a balance change without a ledger row, and the same operation applied twice.

```
// apps/api/src/services/credit.service.ts
export async function apply(tx: Prisma.TransactionClient, i: ApplyInput):
Promise<ApplyResult> {
  // 1. Fast replay check. The unique index in step 3 is the real guard.
  const prior = await tx.creditLedger.findUnique({ where: { idempotencyKey:
i.idempotencyKey } });
  if (prior) return { applied: false, balanceAfter: prior.balanceAfter };

  // 2. One conditional UPDATE. No read-modify-write, so no lost updates.
  const rows = await tx.$queryRaw<{ creditsBalance: number }[]>`
    UPDATE "ClientUser"
       SET "creditsBalance" = "creditsBalance" + ${i.delta}, "updatedAt" =
now()
     WHERE id = ${i.userId} AND "creditsBalance" + ${i.delta} >= 0
 RETURNING "creditsBalance"`;
  if (rows.length === 0) throw new InsufficientCreditsError(i.userId);

  // 3. Ledger row, balanceAfter taken from the same UPDATE. A concurrent
duplicate hits the
  //    unique key here, its whole transaction rolls back (undoing its
balance change), and the
  //    caller treats the unique violation as "already applied" and re-reads
the first result.
  await tx.creditLedger.create({ data: {
    userId: i.userId, amountDelta: i.delta, balanceAfter:
rows[0].creditsBalance,
    type: i.type, idempotencyKey: i.idempotencyKey, reservationId:
i.reservationId,
    paymentId: i.paymentId, reason: i.reason, adminId: i.adminId } });

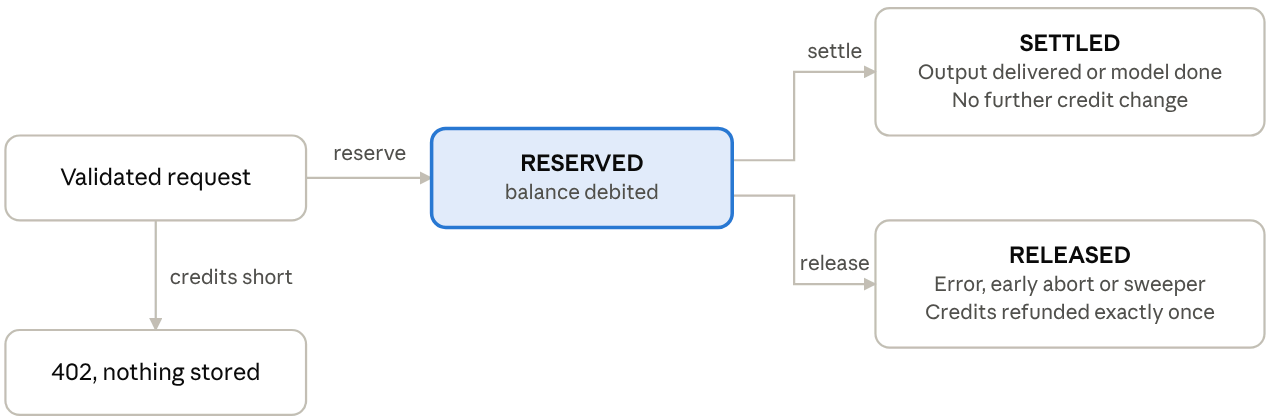
  return { applied: true, balanceAfter: rows[0].creditsBalance };
}
```

AI requests use it through a reservation that is paid for before the Claude call and then settled or released after it.

Operation	What runs in the transaction	Guard against races
`reserve`	Insert the reservation with `ON CONFLICT DO NOTHING`. If a row was inserted, call `apply(-cost)` with key `debit:<userId>:<requestKey>`. If credits are short, roll everything back and return 402	Unique `(userId, requestKey)`; conditional UPDATE
`settle`	`UPDATE CreditReservation SET status = 'SETTLED' WHERE id = $1 AND status = 'RESERVED'`, then write `AiRequestLog` with real tokens and USD cost	Status guard; zero rows means it was already released
`release`	Same guarded UPDATE to `RELEASED` with `RETURNING cost`. If a row came back, call `apply(+cost)` with key `refund:<reservationId>`	Status guard plus unique refund key

Settle, release and the sweeper can all fire for the same reservation. Exactly one wins, because the status change is a guarded UPDATE and the loser changes nothing.

The first guarded update wins and the later one changes nothing



reservation lifecycle · 3 states, 2 terminal

A reservation leaves RESERVED exactly once, to SETTLED or RELEASED, and a request that is short of credits never creates one.

When `reserve` finds an existing reservation for the same key, the answer depends on its state.

Existing reservation	Same payload?	Response
`RESERVED`	Yes	409 `REQUEST_IN_PROGRESS` with `Retry-After`; no second charge
`SETTLED`	Yes	200 with the stored result; no second charge
`RELEASED`	Yes	409 `REQUEST_FAILED_REFUNDED`; the client uses a new key for a fresh attempt
Any	No	422 `KEY_REUSED_WITH_DIFFERENT_PAYLOAD`

The refund rule decides who pays when something goes wrong. Refunding on any client abort would let users stream most of an answer, cancel, and get the credits back while Zibo still pays Anthropic.

Outcome	Credits	Why
Input fails validation	Never reserved	Validation runs before `reserve`
Anthropic error or timeout before any output	Released in full	The user received nothing
Client disconnects before the first output token	Released in full	Abort the upstream call; the cost is negligible
Client disconnects after the first output token	Settled, no refund	Tokens were generated and paid for; this closes the abort-farming loophole
Anthropic error after partial output	Released in full	The failure is on our side
Model declines or hits a safety stop	Settled	The work was done (see open decisions)
Server crashes mid-request	Released by the sweeper	Reservation expires; the user is never charged for a crash

The credit price is fixed when the reservation is created and stored on it, so a rate-card change mid-request cannot make settle and reserve disagree.

AI gate and streaming lifecycle

The gate is a handler wrapper rather than plain middleware, because settlement happens after the Claude call returns. All nine endpoints are registered through it, and a lint rule fails the build if any file imports the Anthropic SDK outside the wrapper.

The wrapper runs these steps in order. Anything that can fail cheaply runs before credits are touched.

1. **Authenticate.** `userId` comes from the verified JWT only. The `req.body.userId` fallback in the master spec lets any caller spend another user's credits, so it is removed.
2. **Validate.** Check the body, image count, image size and history length. A bad request never reserves credits, so it never needs a refund.
3. **Require `Idempotency-Key`.** The client creates one UUID per user action and reuses it only for network-level retries of that same call. A missing key is a 400.
4. **Check the locked-content policy.** If the request references a screen the user cannot view, return 403 `SCREEN_LOCKED` and open the upgrade modal. No charge.
5. **Take a concurrency slot.** A Redis counter per user, capped at a small number such as 3, with a TTL and a release in `finally`. This limits cost exposure and is not needed for correctness, so a crash that leaks a slot only delays the user until the TTL expires.
6. **Reserve credits.** Returns 402, 409, 422 or a stored replay as described in the credit engine.
7. **Call Claude** with an `AbortController`, a per-action `max_tokens` and a hard timeout. A `res.on('close')` listener aborts the upstream call when the client leaves, and records whether any output had already been sent.
8. **Finish.** Settle or release according to the refund table, write `AiRequestLog` with real usage on every outcome, then free the slot.

```
export function metered(action: ActionType, handler: MeteredHandler) {
  return async (req: Request, res: Response) => {
    const userId = req.user?.id;                                    // JWT
only
    if (!userId) return res.status(401).json({ error: 'UNAUTHORIZED' });
    const key = req.header('Idempotency-Key');
    if (!key || !isUuid(key)) return res.status(400).json({ error:
'IDEMPOTENCY_KEY_REQUIRED' });

    const input = validate(action, req.body);                       // 400
```

```
before any charge
    await assertScreensViewable(userId, input);                    // 403
before any charge
    const slot = await Concurrency.acquire(userId);                // 429 if
over the cap
    try {
      const r = await CreditService.reserve({ userId, key, action,
payloadHash: hash(action, input) });
      if (r.kind !== 'reserved') return replyForExisting(res, r);   // 402,
409, 422 or replay

      const ctrl = new AbortController();
      const timer = setTimeout(() => ctrl.abort('TIMEOUT'),
TIMEOUT_MS[action]);
      const state = { outputStarted: false };
      res.on('close', () => { if (!res.writableEnded)
ctrl.abort('CLIENT_ABORT'); });

      let outcome: Outcome;
      try {
        outcome = await handler({ req, res, input, signal: ctrl.signal, state
});
      } catch (err) {
        outcome = classify(err, ctrl.signal, state);                // maps
to settle or release
      } finally {
        clearTimeout(timer);
      }
      await CreditService.finish(r.reservationId, outcome);         // never
throws into the request
    } finally {
      await slot.release();
    }
  };
}
```

Three details matter here.

- **`finish` must not fail the request.** It retries with backoff. If it still fails, the reservation stays `RESERVED`, the sweeper releases it later in the user's favour, and an alert fires. Zibo absorbs one free call instead of charging for a call that failed.

- **Timeouts nest.** The upstream timeout is shorter than the reservation's `expiresAt`, so the sweeper never releases a request that is still legitimately running.
- **Streaming changes the error channel.** Once the first SSE byte is sent, the status code is fixed. Failures after that point go out as an SSE `error` event, so the 402 interceptor only covers failures before streaming starts. That is why every check that can fail is in steps 1 to 6.

Cost control needs limits that the master spec does not define. Before launch, set these per action and record them in the rate card: `max_tokens`, maximum images and bytes per request, a history budget for workspace chat so a long thread cannot cost far more than 10 credits are worth, and an upstream timeout. Set the Anthropic SDK's automatic retries low, since each retry can bill again.

The 402 body should tell the UI which message to show: a free user has used the one-time 50 credits and sees the upgrade prompt, while a PRO user sees the renewal date. The master spec's "Daily credit quota reached" is wrong for both.

Razorpay integration

Three facts from Razorpay's documentation shape this design: webhooks can arrive more than once, and `x-razorpay-event-id` identifies duplicates; failed deliveries are retried with exponential back-off for 24 hours, after which the webhook is disabled; and `subscription.activated` also fires when a subscription recovers from pending or halted, so it is not a payment signal. Razorpay's documentation does not promise event order, so every handler below is written to give the same result in any order.

Creating the subscription

`POST /billing/subscribe` takes a plan code and a currency. The server, not the client, looks up the `PlanPrice` row.

1. Take a per-user Postgres advisory lock (`pg_advisory_xact_lock`) so two clicks serialise.
2. Look for a live subscription. If it is `ACTIVE`, return 409 `ALREADY_SUBSCRIBED`. If it is a recent `INCOMPLETE` one, confirm with Razorpay that it is still unpaid and return the same `subscription_id`.
3. Otherwise create it at Razorpay with `notes: { userId, priceId }`, then insert the local `UserSubscription` as `INCOMPLETE`. The partial unique index is the backstop if the lock is ever bypassed.

4. If the process dies between Razorpay creating the subscription and the local insert, an unpaid orphan remains at Razorpay. It is harmless, and the reconciler cancels orphans older than a day using the `notes.userId` tag.

Razorpay requires `total_count`, the number of billing cycles. Choose a large number such as 120 and alert when a subscription nears completion (see edge cases).

Checkout and return

The client opens Razorpay Checkout with the `subscription_id`. The success handler returns a payment id, subscription id and signature, which the client posts to `POST /billing/verify`. The server checks the signature (confirm the exact string Razorpay signs for subscriptions against its docs) and returns `PROCESSING`. It grants nothing.

The UI shows "Activating PRO" and polls `GET /billing/status` every two seconds. If the webhook has not landed after about 15 seconds, the status endpoint takes the **fast path**: it fetches the payment and subscription from Razorpay, which Razorpay recommends when a webhook is late, and calls the same `fulfilCharge` function the webhook uses. Both paths share the ledger key `pay:<razorpayPaymentId>`, so whichever runs second changes nothing.

Webhook pipeline

The route stays mounted before `express.json()` so the raw bytes survive. In order:

1. Verify `HMAC-SHA256(rawBody, secret)` against `x-razorpay-signature` with `crypto.timingSafeEqual`, comparing equal-length buffers. Accept two secrets during a rotation window. A mismatch returns 400.
2. Require `x-razorpay-event-id`. Unknown event types are recorded and acknowledged with 200.
3. Open one transaction and insert into `PaymentWebhookEvent`. A unique violation means the event was already processed, so return 200. If a concurrent duplicate is mid-flight, its insert waits on the unique index and then fails the same way.
4. Resolve the user from `razorpaySubId`, falling back to `notes.userId`, and take the per-user advisory lock.
5. Dispatch to the handler below. It writes to the same transaction and ends with `recomputeTier`.

6. Commit and return 200. On any error, roll back (which also removes the event row, so Razorpay's retry reprocesses it), write a `PaymentWebhookFailure` row outside the transaction, and return 500.

No network call and no email happens inside this transaction. Notifications go through an outbox that a worker drains after commit.

Event handlers

Credits are granted in exactly one place, keyed by payment id. Status events only move status.

Event	Effect
`subscription.authenticated`	Record the raw status. No credits, no tier change
`subscription.activated`	Set `ACTIVE` and the period dates; call `recomputeTier`. No credits
`subscription.charged`	`fulfilCharge`: upsert `PaymentTransaction` by `razorpayPaymentId`; if it is new and the amount matches the plan price, grant `plan.monthlyCredits` under key `pay:<paymentId>`; set period dates, `ACTIVE` and `entitledUntil = currentPeriodEnd`
`payment.failed`	Record a `FAILED` transaction by payment id and enqueue a notification. No state change
`subscription.pending`	`PAST_DUE`; `entitledUntil = currentPeriodEnd + grace`
`subscription.halted`	`HALTED`; `entitledUntil = now()`; the user drops to FREE and keeps the credits already granted
`subscription.paused`, `subscription.resumed`	`PAUSED` keeps access until period end; `resumed` returns to `ACTIVE`
`subscription.cancelled`	`CANCELLED`; `entitledUntil = currentPeriodEnd` for cancel-at-cycle-end, otherwise `now()`

Event	Effect
`subscription.completed`	`COMPLETED`; `entitledUntil = currentPeriodEnd`; page the team, since this should not happen early
`subscription.updated`	Refresh plan, price and period from the payload. No credits
`refund.processed`	Match the refund to a payment transaction that granted credits, mark it refunded and claw back unspent credits per policy. Ignore refunds with no such transaction, such as the small card-authorisation token that Razorpay refunds automatically

Two more rules apply to all handlers. Status events compare the payload timestamp with `lastEventAt` and skip the state change if they are older, but charge and refund events always apply, since they record facts. And in Razorpay's test mode, confirm that the first payment of your plan configuration emits `subscription.charged`. If it does not, call `fulfilCharge` from the event that carries the first payment as well. Because the key is the payment id, calling it from two event types still grants once.

Subscribe to these events in the Razorpay dashboard, plus the dispute events, which go to a manual review queue rather than an automatic handler.

Subscription state machine and tier derivation

Access is decided by one timestamp, `entitledUntil`, rather than by branching on status. Handlers set it per the table above, and tier follows from it.

```
function deriveTier(user: ClientUser, sub: UserSubscription | null, now:
Date): UserTier {
  if (user.tierOverride && (!user.tierOverrideUntil || user.tierOverrideUntil
> now)) {
    return user.tierOverride;                                  // admin VIP
wins
  }
  if (sub?.entitledUntil && sub.entitledUntil > now) return 'PREMIUM';
  return 'FREE';
}
```

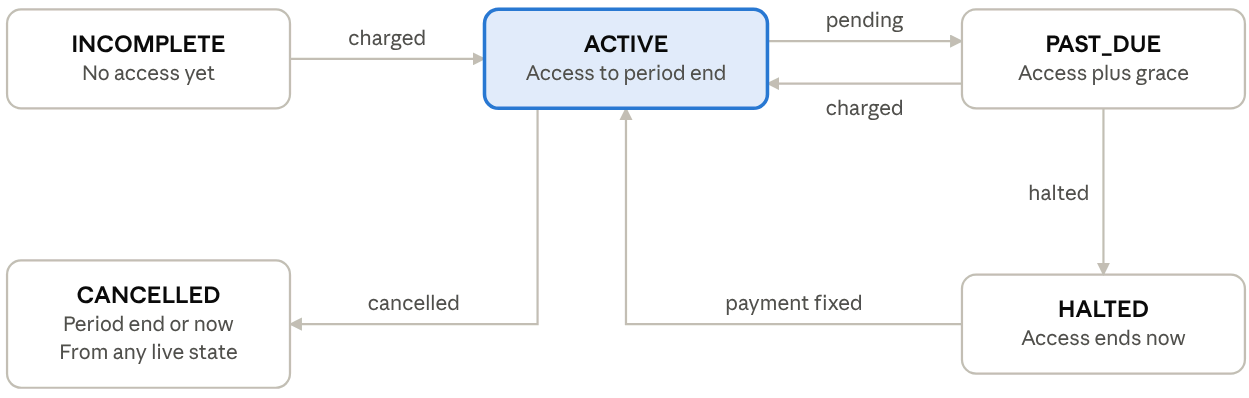
Two consequences are easy to miss.

- **Nothing fires when a paid period simply ends.** A subscription cancelled at cycle end produces a `cancelled` event weeks before access should stop. The entitlement-expiry worker therefore re-derives tier for every user whose `entitledUntil` has just passed.
- **Request-time checks use the timestamp, not the cached column.** `canViewScreen` and the AI gate call `deriveTier` with the current time, so a lagging worker cannot extend access. `ClientUser.tier` exists for admin lists and analytics, and the tier is never read from the JWT.

Razorpay's statuses map to ours as follows. The retry timing for failed charges differs by payment method, so the plan does not promise a fixed three-day grace. Grace is a configurable number of days after period end, and Razorpay's own `halted` event is what ends it.

Razorpay	Ours	Access (`entitledUntil`)	Credits
`created` , `authenticated`	`INCOMPLETE`	None	None until the first charge is confirmed
`active` after a charge	`ACTIVE`	Current period end	Monthly grant on each `charged` payment
`pending` (charge failed, retrying)	`PAST_DUE`	Period end plus configured grace	No grant until a retry succeeds, then the normal grant for that cycle
`halted`	`HALTED`	Ends now	Existing balance kept; the missed cycle is not charged again unless support charges it manually
`paused`	`PAUSED`	Period end	None while paused
`cancelled`	`CANCELLED`	Period end for cancel-at-cycle-end, otherwise now	Existing balance kept
`completed` , `expired`	`COMPLETED` , `EXPIRED`	Period end	Existing balance kept

Only a successful charge moves a subscription to ACTIVE



Paused, completed and expired follow the table above.

subscription states · 5 states, 6 transitions

Access is extended only by a confirmed charge and cut by `halted` or a cancellation, which is why `activated` alone never grants credits.

When a halted subscription recovers, Razorpay does not re-attempt the invoices it skipped, so the user receives credits only for charges that actually succeed. A UPI subscription paused by the customer can only be resumed by the customer, so the UI must say "resume it in your UPI app" instead of showing a Resume button.

Paywall: gating every route that returns screen data

The master spec gates one controller and still returns `imageUrl` for locked screens, so the full image is one click away in the network tab. The fix is one policy, enforced at every place that can return screen content.

```
// canViewScreen is the only place that decides. Routes and AI loaders call
it, never inline checks.
function canViewScreen(viewer: Viewer | null, app: App, screenIndex: number,
now: Date): boolean {
  if (!app.isPremium) return true;
  if (viewer && deriveTier(viewer.user, viewer.sub, now) === 'PREMIUM')
return true;
  return screenIndex < (app.previewLimit ?? 4);               // ?? so an
admin value of 0 means 0
}
```

For a locked screen the response carries a pre-generated heavily blurred, low-resolution derivative and the screen's name and number. It never carries the original URL, `uxAnalysis`, `keyHighlights`, OCR text or tags that describe the content. Originals live in a private bucket and are served only as short-lived signed URLs issued after `canViewScreen` passes.

Every route below must call the policy. Gating only the app-detail route is the main gap in the master spec.

Surface	Leak if ungated	Rule
App detail, screen list, screen detail, flows	Original image URL or analysis	Locked derivative plus redacted fields
Search and browse cards	Thumbnails and snippets of locked screens	Same redaction on every card

Surface	Leak if ungated	Rule
Similar-screen and embedding endpoints	Analysis text returned by retrieval	Filter results by `canViewScreen` after retrieval
AI endpoints: insights, compare, inspect, improve-design, workspace chat RAG	Claude's answer reveals locked content	Return 403 `SCREEN_LOCKED` before reserving credits; drop locked screens from RAG context
Boards and workspaces	A PRO user saves a premium screen, a free teammate opens the board	Resolve visibility per viewer at read time, never at save time
Download and export	Full images in a file	PRO only, checked server-side
Public pages and Open Graph images	Social previews expose the original	Use the locked derivative
CDN and HTTP caches	One user's unlocked JSON served to everyone	`Cache-Control: private` and `Vary: Authorization` on every gated response

Premium status can change after people have saved content, so the rules must also hold over time. When an app flips from free to premium, nobody is grandfathered. When a PRO user downgrades, boards they created stay visible to them as redacted. When `previewLimit` exceeds the screen count, no lock is shown.

A CI leak test enforces this. It logs in as a free user, walks every route above for a premium app, and fails if any response body contains a locked image URL or a string from locked analysis.

Background workers

Six jobs repair what the request path cannot. They run in a separate worker process from the API, with one scheduler instance. Use BullMQ's job-scheduler API, since the older repeatable-job option is deprecated; check the installed version, because the master spec's "v6" should be confirmed. Every job is idempotent, so a second scheduler by accident causes no double refund.

Job	Cadence	What it does	Safety
Reservation sweeper	1 minute	Selects `RESERVED` rows past `expiresAt` and releases each with reason `EXPIRED`	`FOR UPDATE SKIP LOCKED`, batches of 100; same guarded `release` as the request path
Webhook failure retrier	5 minutes	Re-runs unresolved `PaymentWebhookFailure` payloads through the normal handler	Event id and payment-id keys make reruns harmless; stops after a set number of attempts and alerts
Subscription reconciler	10 minutes	Fetches from Razorpay only the subscriptions that can be wrong: `INCOMPLETE` older than 15 minutes, `PAST_DUE`, and `ACTIVE` whose period ended more than 2 hours ago. Also finds paid invoices with no `PaymentTransaction` and cancels orphans	Feeds drift through the same handlers as a synthetic event with a deterministic id; rate-limited; a daily paged full sweep catches anything else
Entitlement expiry	5 minutes	Re-derives tier for users whose `entitledUntil` or `tierOverrideUntil` just passed	Pure function of stored data
Balance verifier	Daily, 00:00 UTC	Compares each balance with its ledger sum and checks the `balanceAfter` chain	Alerts only; never auto-corrects money
Cost and margin report	Weekly	Joins `AiRequestLog.costUsd` with credits charged per action	Read only

The verifier's core query is set-based, so it scales to every user in one statement:

```
SELECT u.id, u."creditsBalance" AS balance, COALESCE(SUM(l."amountDelta"), 0)
AS ledger_sum
FROM "ClientUser" u
LEFT JOIN "CreditLedger" l ON l."userId" = u.id
GROUP BY u.id
HAVING u."creditsBalance" <> COALESCE(SUM(l."amountDelta"), 0);
```

The reconciler is what makes a missed webhook a non-event. Razorpay disables a webhook after 24 hours of failed delivery, so without it a long outage would leave paying users on FREE until someone noticed. Reconciler and webhook meet in the same handlers, which is why drift repaired by one cannot be applied again by the other.

Race condition catalog

Twenty-two races, each with the fix already placed in the design above. IDs are used by the test matrix.

ID	Race	What goes wrong	Fix
R1	Two AI requests at once with credits for one	Both pass a read-check and the balance goes negative	Conditional `UPDATE ... WHERE balance >= cost`; DB `CHECK` as backstop
R2	Client retries after a timeout	Charged twice for one action	`Idempotency-Key` with unique `(userId, requestKey)`; replay rules
R3	Two deliveries of the same key at the same instant	Two reservations, two debits	`ON CONFLICT DO NOTHING` on the reservation; unique ledger key rolls back the loser
R4	Settle, release and sweeper on one reservation	Double refund, or a refund after a settle	Guarded status `UPDATE`; refund key `refund:<reservationId>`
R5	Sweeper fires while a request still streams	User gets a free call	`expiresAt` longer than the upstream timeout; `settle_after_release` metric
R6	Same webhook delivered twice concurrently	Double credit grant	Event-id insert inside the transaction; payment-id ledger key
R7	Webhook and browser fast path fulfil one payment	Double grant	Both call `fulfilCharge`; ledger key `pay:<paymentId>` is unique

ID	Race	What goes wrong	Fix
R8	Webhooks arrive out of order	Status moves backwards, for example `ACTIVE` overwritten by an old `pending`	`lastEventAt` guard for status events; charges apply in any order; reconciler corrects the rest
R9	`activated` and `charged` both grant credits	Double grant on the first payment	Only payment fulfilment grants, keyed by payment id
R10	Double-click on Start With PRO	Two Razorpay subscriptions, two mandates	Advisory lock, reuse of the unpaid `INCOMPLETE` one, partial unique index
R11	Second tab or device pays while already `ACTIVE`	Two live subscriptions, double billing	409 before creation; if a second one still becomes live, cancel the newer at Razorpay and refund its first charge, then alert
R12	User returns from checkout before the webhook	UI still shows locked	Polling plus the API fast path
R13	Admin credit change during an AI debit	Lost update	Both use `apply()`, one atomic `UPDATE`; no read-modify-write
R14	Admin override while a webhook recomputes tier	Webhook overwrites the VIP	Override is its own field; `tier` is derived; advisory lock in `recomputeTier`
R15	User cancels just before the renewal debit	They are charged, then cancelled	Grant credits for the charge (they paid), honour the cancellation after, and refund on request
R16	Refund lands after credits were spent	Clawback would push balance below zero	Claw back `min(granted, balance)`; log the shortfall; flag the account
R17	Two first requests create the user	Welcome grant applied twice	Ledger key `welcome:<userId>` plus `welcomeGrantedAt`; user create is an upsert

ID	Race	What goes wrong	Fix
R18	Two scheduler instances run one job	Duplicate work	One scheduler; `SKIP LOCKED`; every action idempotent
R19	Tier cached in the JWT after downgrade	Access outlives the subscription	Request-time `deriveTier` from the database
R20	Plan price edited during checkout	User pays one price, is recorded at another	Prices are immutable; a change creates a new `PlanPrice`; `priceId` stored at subscribe
R21	App server clock differs from the database	Entitlement ends early or late	Compare against database `now()` in SQL
R22	Webhook secret rotated	Valid events fail signature	Accept two secrets during the rotation window

Edge case catalog

Races are about timing. These are the situations that are not, grouped by where they arise. Each has a decided behaviour or points to an open decision.

Payments and checkout

- **User pays, then closes the tab.** Nothing is lost: the webhook fulfils it, and a confirmation email goes out from the outbox.
- **UPI first debit is slow.** Razorpay notes that a retry only happens once the previous attempt is confirmed or rejected, which can take more than a day. The subscription stays `INCOMPLETE` and the UI says "waiting for your bank". No credits until a charge is confirmed.
- **Webhook endpoint down for more than a day.** Razorpay disables the webhook and emails the alert address. The reconciler repairs state, and a daily heartbeat alerts if no event of any kind arrived in 24 hours.
- **Amount above the silent-debit limit.** UPI Autopay needs extra authentication on every debit above 15,000 rupees. At 1,499 rupees this does not apply today, but the admin plan form must warn on any price near the limit, since a price rise would break silent renewals. Limits vary by merchant category, so confirm the figure for your account.

- **Card expired or bank mandate revoked.** The subscription goes `pending`, then `halted`. The user fixes it through Razorpay's hosted link, and recovery arrives as `activated`. Missed cycles are not charged again.
- **Customer pauses from their UPI app.** Only they can resume it. The UI explains this instead of offering a button.
- **Refund requested.** Done in the Razorpay dashboard; the `refund.processed` webhook triggers the clawback policy. Disputes go to a manual review queue and flag the account. Razorpay refunds the small card-authorisation token automatically, and that refund must never trigger a clawback.
- **Tax and invoice.** Whether the price includes GST, and what invoice the user receives, is an open decision to confirm with an accountant.
- **International cards.** Confirm with Razorpay that international subscriptions and USD plans are enabled for the account. If not, launch INR-only.

Subscriptions

- **Resubscribe after cancel or halt.** Creates a new Razorpay subscription. The old one is terminal. The credit balance carries over.
- **Cancel.** Default is cancel at cycle end, so the user keeps PRO and their credits until the period ends. Immediate cancellation is not offered.
- **Subscription reaches `total_count`.** It completes, which looks like an involuntary cancellation to the user. Alert the team well before the last cycle and ask the user to renew with consent.
- **Plan change from Pro to Agency.** Not in version one. An admin handles it manually until proration rules are decided. Razorpay can update only card-authorised subscriptions, not UPI or e-mandate ones, so for a UPI user a plan change means cancelling and creating a new subscription.
- **Account deletion with a live subscription.** Cancel at Razorpay first, then anonymise the user. Never hard-delete rows in money tables.
- **Team or VIP access with no payment.** Use `tierOverride` with a reason and an optional end date.

Credits

- **Balance exactly equals the cost.** Allowed, since the check is `>=`.
- **Free to PRO.** Leftover welcome credits stay and the monthly grant is added.

- **PRO to FREE.** The balance stays and nothing more is granted. Whether unused credits roll over without limit is an open decision.
- **Rate card changes while requests are in flight.** The cost is stored on the reservation, so it is unaffected.
- **Admin deduction larger than the balance.** Rejected with the current balance shown, never clamped silently.
- **Anthropic SDK retries.** Keep automatic retries low and log the real cost of every attempt.

Admin

- Every admin write needs a role check, a mandatory reason, a request key for idempotency and an `AdminAuditLog` row with before and after values.
- Bulk adjustments take a CSV and show a dry-run diff before applying.

Abuse and security

- **Welcome-credit farming.** Grant the 50 credits only after email verification, not at signup. Normalise Gmail dots and plus aliases, block disposable domains, and alert on bursts per IP or device. Phone verification is the stronger option if farming appears.
- **Webhook endpoint.** Signature check, body size limit, rate limit, and an optional allowlist of Razorpay's published webhook IPs.
- **Card data.** Never stored or logged; Razorpay Checkout handles it. Scrub payloads in logs of contact details beyond what support needs.
- **Retention.** Financial rows are kept for the period your accountant specifies, and user deletion anonymises rather than removes them.

Phased implementation plan

The order puts the ledger first, because every other part writes to it, and the paywall in parallel, because it is independent and closes the largest content leak. A phase is done only when its exit criteria pass, not when its code merges. Sizes are relative: S is a few days of work, M about a week or two, L more.

Phase	Scope	Exit criteria	Needs	Size
P0. Decisions and setup	Settle the open decisions below. Create a Razorpay test account, plans per currency, the webhook URL and event list, and secrets	Decisions recorded; a signed test webhook reaches staging	None	S
P1. Ledger core	Migration with constraints, `CreditService.apply`, welcome grant in the user-create transaction, backfill of existing balances, verifier query	R1, R3, R13 and R17 pass with 100 parallel callers; verifier is clean on a copy of production data	P0	M
P2. AI gate	Reservation lifecycle, `metered` wrapper on all nine endpoints, concurrency cap, sweeper, cost caps, `AiRequestLog`	R2 to R5 pass; killing the process mid-stream, aborting, and an upstream 500 each end with the correct balance; lint blocks direct SDK use	P1	L
P3. Paywall hardening	`canViewScreen`, private bucket and signed URLs, locked derivatives, every route in the paywall table, AI context filtering, cache headers	Leak test passes in CI; manual audit of the route table signed off	None	M
P4. Razorpay subscriptions	Subscribe endpoint, verify and polling, webhook pipeline and handlers, `PaymentTransaction`, `entitledUntil` and `recomputeTier`, dead-letter table, secret rotation	Scripted test-mode scenarios each end with the expected credits and tier: card and UPI happy paths, duplicate delivery, shuffled order, failure to halted to recovery, cancel at cycle end	P1	L

Phase	Scope	Exit criteria	Needs	Size
P5. Workers	Reconciler, failure retrier, entitlement expiry, verifier alerts, separate worker process	Drill: block webhooks for an hour in staging; the reconciler repairs everything with no manual database edit	P4	M
P6. Frontend	Upgrade modal flow, activation polling, tier-aware 402 banner, 403 `SCREEN_LOCKED` handling, PRO tags and padlocks	End-to-end tests for upgrade, failed payment and locked screen paths	P2, P3, P4	M
P7. Admin suite	Credit adjust, tier override, plans, paid-app toggle, audit views	Every admin write is role-checked, idempotent and audited	P1	M
P8. Launch	Feature flags, shadow mode, canary, runbook, alerts live, rollback plan	Shadow mode shows zero ledger drift for seven days; every alert fired once on purpose	All	M

Shadow mode is the safest way into P8. Run `reserve` and `settle` for real but have the gate log "would have blocked" instead of returning 402. A week of that compares predicted deductions with actual Anthropic cost per action, and exposes margin or rate-card errors before any user is blocked. Keep `billing_enabled` and `ai_gate_enforced` as separate flags so either can be rolled back alone.

Test matrix

Unit tests on mocks will not catch these bugs, because they live in Postgres locking and event ordering. Run every test below against a real Postgres, for example in a container, with a fake clock injected into entitlement code and signed Razorpay fixtures for webhooks.

Test	Setup	Assertion	Covers
Parallel debit	Balance 10, cost 10, 50 simultaneous requests with different keys	Exactly one succeeds, 49 get 402, balance 0, one ledger row	R1, I2
Retry storm	Same key sent 20 times in parallel	One debit; others get 409 or the stored replay	R2, R3
Settle, release and sweep race	Fire all three at one reservation in random order, repeated 1,000 times	One terminal status; refund at most once; ledger sum equals balance	R4, I1
Crash mid-stream	Kill the process after reserve, before finish	Sweeper releases after `expiresAt`; balance restored	R5, R18
Abort policy	Abort before and after the first token	Refund before, no refund after, real cost logged in both	Refund table
Duplicate webhook	Same signed event delivered 10 times in parallel	One `PaymentWebhookEvent`, one grant of the plan's credits	R6, I3
Fast path versus webhook	Run `fulfilCharge` from both for one payment id	One grant	R7
Shuffled events	Replay a full lifecycle stream in every order	Same final credits and `entitledUntil`	R8, R9
Failure and recovery	`charged`, then `pending`, `halted`, then `activated` and `charged`	Tier follows the table; no grant for the missed cycle	State table
Cancel at cycle end	Cancel, then advance the fake clock past period end	Tier is FREE only after the period ends; expiry worker fixes it	R19, R21
Subscribe double-click	Two simultaneous `POST /billing/subscribe`	One Razorpay subscription and one local row	R10, R11
Refund after spend	Refund a charge whose credits are partly used	Clawback capped at the balance; no negative	R16

Test	Setup	Assertion	Covers
Admin versus AI	Admin adjusts while 20 AI requests run	Ledger sum equals balance	R13
Random interleaving	Property test: random sequences of all operations	I1 to I3 hold after every sequence	All invariants
Leak test	Free user walks every gated route on a premium app	No locked URL or analysis text appears	I6
Gateway drill	Block webhooks in staging, make real test payments	Reconciler repairs state within its interval	I8

The random-interleaving test earns its cost. It generates sequences of debits, refunds, webhooks and admin edits, runs them concurrently, and checks the invariants afterwards. It finds the races this document did not think of.

Observability, alerts and runbook

Money bugs are found by users unless something finds them first. These signals are emitted from day one, and the thresholds are starting values to tune during shadow mode.

Signal	Alert when	Severity
Verifier discrepancy (balance differs from ledger sum)	Any user, any run	Page
Negative balance or negative `balanceAfter`	Any row; the database should make this impossible	Page
`PaymentWebhookFailure` unresolved	More than 15 minutes; page after 6 hours, well inside Razorpay's 24-hour retry window	Warn, then page
No webhook of any kind received	24 hours while live subscriptions exist	Page
Webhook signature failures	A sudden rise	Warn

Signal	Alert when	Severity
Reconciler found drift	Any repair, since it means a webhook was missed	Warn
Reservations `RESERVED` past `expiresAt`	More than 5 minutes after expiry	Warn
`settle_after_release` count	Any	Warn; the timeouts are misaligned
`finish` failures after retries	Any	Warn
Realised margin per action	Below target in the weekly report	Review
402 or 403 `SCREEN_LOCKED` rate	Sharp change after a deploy	Info, usually a client bug

The runbook covers the five incidents most likely to reach support.

Symptom	First checks	Fix
"I paid but I am not PRO"	`PaymentTransaction` by payment id; `PaymentWebhookEvent` and `PaymentWebhookFailure`; the subscription at Razorpay	Run the idempotent admin reconcile for that user; if the payment is real, `fulfilCharge` grants once
Verifier reports a mismatch	The user's ledger in order, looking for a gap or a duplicate	Never edit the balance. Add a corrective ledger row through the admin endpoint with a reason, then find the root cause
Webhook disabled email from Razorpay	Endpoint health and the error that caused the failures	Fix, re-enable in the dashboard, then run a full reconciler sweep
"I was charged twice"	Two live subscriptions for the user	Cancel the newer one at Razorpay, refund its charge, record it

Symptom	First checks	Fix
"My AI request failed and I lost credits"	The reservation row and its release reason	If it settled although it failed, fix the classifier and refund through admin with a reason

The admin reconcile endpoint, `POST /admin/billing/reconcile/:userId`, is part of P7. It runs the same code as the worker for one user, so support can repair a case without a database session.

Open decisions

Twelve choices change the code or the numbers, and each has a recommended default so work can start. P0 closes them.

#	Decision	Recommendation	Why it matters
1	What a credit is worth	Define it in dollars first, then recompute the rate card	At $19 for 2,500 credits a credit is about $0.0076, so the spec's margins (58 to 70%) understate the real ones; the table implies about $0.003
2	Do unused monthly credits roll over	Carry over up to twice the monthly grant, expire the excess with a `ROLLOVER_EXPIRY` ledger row	Unlimited rollover builds a liability; none upsets paying users
3	Abort after the first output token	No refund	Closes abort-farming; a refund on abort lets users get free output
4	Model declines or safety stop	Charge	The cost was incurred; refunds invite probing
5	Grace period for a failed renewal	Keep PRO until Razorpay sends `halted`, capped at a configured number of days	Retry timing varies by payment method, so a fixed three days would be wrong

#	Decision	Recommendation	Why it matters
6	Refund policy	Full refund of the latest charge within a stated window cancels the subscription and claws back unspent credits	Without a rule, support invents one per case
7	GST and price display	Confirm with an accountant whether 1,499 rupees is inclusive	Affects the invoice, the plan amount and net revenue
8	USD plans	Launch INR-only unless Razorpay confirms international subscriptions on the account	A USD `razorpayPlanId` that cannot be charged breaks checkout abroad
9	Email verification before the welcome grant	Required	Cheapest defence against multi-account farming
10	Credit packs	Defer to version two	Packs need credit buckets with separate expiry, which the single balance cannot express
11	Plan changes such as Pro to Agency	Admin-only in version one	Proration and credit rules are a project of their own, and Razorpay cannot update UPI subscriptions in place
12	`total_count` for subscriptions	120 monthly cycles with an alert before the end	Razorpay requires a finite count, up to a 100-year maximum duration; completion looks like a surprise cancellation

Sources

Razorpay documentation, checked on 2026-11-11. Billing behaviour changes, so confirm each point against the current pages before building.

- Webhooks: duplicate delivery and the event id header (search excerpt)
- Webhooks: 24-hour retry window and disabling (search excerpt)
- Webhooks: fetch from the API when an event is late (search excerpt)
- Subscription webhook events (page opened)
- Payment retries (page opened)
- Subscription states (search excerpt)
- Subscriptions FAQs: immutable plans, card-only updates, UPI pause, authorisation token (page opened)

Not checked against documentation: the exact string signed in the checkout handler for subscriptions, the payload timestamp field used for `lastEventAt`, the BullMQ job-scheduler API in the installed version, whether Razorpay international subscriptions are enabled on your account, and the UPI Autopay limit for your merchant category.